

AJAX & jQuery

Web Programming Assignments 20–29

Minimal Working Code · Standard Approach · Exam Ready

Standard Approach used across all assignments:

- **Vanilla JS (20–25):** XMLHttpRequest → open → setRequestHeader → onload/onerror → send
- **jQuery (26–29):** \$.ajax({ url, method, success, error })
- Spinner div toggled via display:none/block (show/hide)
- Error div for user-friendly messages
- No <form> submissions — all interactions prevent page reload
- Mock API: JSONPlaceholder (<https://jsonplaceholder.typicode.com>)

Assignment 20: Weather App (Vanilla JS + AJAX)

Note: Replace 'YOUR_API_KEY' with a free key from openweathermap.org

```
<!DOCTYPE html>
<html>
<head><title>Weather App</title>
<style>
  body { font-family: Arial; max-width: 400px; margin: 50px auto; padding: 20px; }
  input, button { padding: 8px; margin: 5px 0; width: 100%; box-sizing: border-box; }
  #result { margin-top: 15px; padding: 10px; border: 1px solid #ddd; display: none; }
  #error { color: red; }
  #spinner { display: none; color: gray; }
</style>
</head>
<body>
<h2>Weather App</h2>
<input type="text" id="city" placeholder="Enter city name">
<button onclick="getWeather()">Get Weather</button>
<div id="spinner">Loading...</div>
<div id="error"></div>
<div id="result">
  <p>Temperature : <span id="temp"></span> C</p>
  <p>Humidity    : <span id="humidity"></span> %</p>
  <p>Condition    : <span id="condition"></span></p>
  <p>Wind Speed   : <span id="wind"></span> m/s</p>
</div>

<script>
function getWeather() {
  var city    = document.getElementById('city').value.trim();
  var API_KEY = 'YOUR_API_KEY';          // Replace with your key
  var error   = document.getElementById('error');
  var result  = document.getElementById('result');
  var spinner = document.getElementById('spinner');

  error.textContent = '';
  result.style.display = 'none';

  if (!city) { error.textContent = 'Please enter a city name.'; return; }

  spinner.style.display = 'block';

  var xhr = new XMLHttpRequest();
  xhr.open('GET',
    'https://api.openweathermap.org/data/2.5/weather?q=' + city +
    '&units=metric&appid=' + API_KEY);

  xhr.onload = function () {
    spinner.style.display = 'none';
    if (xhr.status === 200) {
      var d = JSON.parse(xhr.responseText);
      document.getElementById('temp').textContent    = d.main.temp;
      document.getElementById('humidity').textContent = d.main.humidity;
      document.getElementById('condition').textContent = d.weather[0].description;
      document.getElementById('wind').textContent    = d.wind.speed;
      result.style.display = 'block';
    } else if (xhr.status === 404) {
      error.textContent = 'City not found. Please try again.';
    } else {
      error.textContent = 'API error. Please try again later.';
    }
  };

  xhr.onerror = function () {
```

```
        spinner.style.display = 'none';
        error.textContent = 'Network error. Check your connection.';
    };

    xhr.send();
}
</script>
</body>
</html>
```

Assignment 21: Live User Search (Vanilla JS + AJAX)

```
<!DOCTYPE html>
<html>
<head><title>Live User Search</title>
<style>
  body    { font-family: Arial; max-width: 400px; margin: 50px auto; padding: 20px; }
  input    { padding: 8px; width: 100%; box-sizing: border-box; }
  #dropdown{ border: 1px solid #ccc; max-height: 200px; overflow-y: auto; display: none; }
  .item     { padding: 8px; cursor: pointer; }
  .item:hover { background: #f0f0f0; }
  #loading  { color: gray; font-size: 12px; }
  #error    { color: red; }
</style>
</head>
<body>
<h2>Live User Search</h2>
<input type="text" id="search" placeholder="Search users..."
        oninput="searchUsers(this.value)">
<div id="loading"></div>
<div id="dropdown"></div>
<div id="error"></div>

<script>
var timer;

function searchUsers(query) {
  var dropdown = document.getElementById('dropdown');
  var loading  = document.getElementById('loading');
  var error    = document.getElementById('error');

  dropdown.style.display = 'none';
  error.textContent = '';

  if (!query.trim()) { loading.textContent = ''; return; }

  clearTimeout(timer);          // debounce: wait 300ms after typing stops
  timer = setTimeout(function () {
    loading.textContent = 'Loading...';

    var xhr = new XMLHttpRequest();
    xhr.open('GET', 'https://jsonplaceholder.typicode.com/users');

    xhr.onload = function () {
      loading.textContent = '';
      if (xhr.status === 200) {
        var users    = JSON.parse(xhr.responseText);
        var filtered = users.filter(function (u) {
          return u.name.toLowerCase().indexOf(query.toLowerCase()) !== -1;
        });
        dropdown.innerHTML = filtered.length
          ? filtered.map(function (u) {
              return '<div class="item">' + u.name + ' (' + u.email + ')</div>';
            }).join('')
          : '<div class="item">No users found</div>';
        dropdown.style.display = 'block';
      } else {
        error.textContent = 'Failed to fetch users.';
      }
    }
  }, 300);
};

xhr.onerror = function () {
  loading.textContent = '';
};
```

```
        error.textContent = 'Network error.';
    };

    xhr.send();
}, 300);
}
</script>
</body>
</html>
```

Assignment 22: Dynamic To-Do List (AJAX + DOM Manipulation)

```
<!DOCTYPE html>
<html>
<head><title>To-Do List</title>
<style>
  body    { font-family: Arial; max-width: 500px; margin: 50px auto; padding: 20px; }
  input   { padding: 8px; width: 70%; }
  button  { padding: 8px 12px; }
  li      { padding: 6px; margin: 4px 0; background: #f9f9f9;
            display: flex; justify-content: space-between; }
  .del-btn{ background: red; color: white; border: none; padding: 2px 8px; cursor: pointer; }
  #spinner{ display: none; color: gray; }
  #error   { color: red; }
</style>
</head>
<body>
<h2>To-Do List</h2>
<input type="text" id="taskInput" placeholder="New task...">
<button onclick="addTask()">Add</button>
<div id="spinner">Loading...</div>
<div id="error"></div>
<ul id="taskList"></ul>

<script>
var API = 'https://jsonplaceholder.typicode.com/todos';

function showSpinner(s) { document.getElementById('spinner').style.display = s ? 'block' : 'none'; }
function setError(m)    { document.getElementById('error').textContent = m; }

function ajax(method, url, data, cb) {
  var xhr = new XMLHttpRequest();
  xhr.open(method, url);
  xhr.setRequestHeader('Content-Type', 'application/json');
  xhr.onload = function () { cb(xhr.status, xhr.responseText); };
  xhr.onerror = function () { cb(0, null); };
  xhr.send(data ? JSON.stringify(data) : null);
}

function addToUI(id, title) {
  var li = document.createElement('li');
  li.id = 'task-' + id;
  li.innerHTML = title +
    ' <button class="del-btn" onclick="deleteTask(' + id + ')">Delete</button>';
  document.getElementById('taskList').appendChild(li);
}

function fetchTasks() {
  showSpinner(true);
  ajax('GET', API + '?_limit=5', null, function (status, res) {
    showSpinner(false);
    if (status === 200)
      JSON.parse(res).forEach(function (t) { addToUI(t.id, t.title); });
    else
      setError('Failed to fetch tasks.');
```

```
    ajax('POST', API, { title: title, completed: false, userId: 1 },
    function (status, res) {
        showSpinner(false);
        if (status === 201) {
            var task = JSON.parse(res);
            addToUI(task.id, task.title);
            document.getElementById('taskInput').value = '';
        } else {
            setError('Failed to add task.');
```

Assignment 23: Country-State-City Dropdown (Event-Driven AJAX)

Uses predefined JSON (mock API) — XMLHttpRequest simulates the API call pattern.

```
<!DOCTYPE html>
<html>
<head><title>Cascading Dropdown</title>
<style>
  body { font-family: Arial; max-width: 400px; margin: 50px auto; padding: 20px; }
  select { padding: 8px; width: 100%; margin: 8px 0; }
  #error { color: red; }
</style>
</head>
<body>
<h2>Country - State - City</h2>
<select id="country" onchange="loadStates()">
  <option value="">-- Select Country --</option>
  <option value="IN">India</option>
  <option value="US">USA</option>
</select>
<select id="state" onchange="loadCities()" disabled>
  <option value="">-- Select State --</option>
</select>
<select id="city" disabled>
  <option value="">-- Select City --</option>
</select>
<div id="error"></div>

<script>
// Mock data - simulates API response
var mockData = {
  states: {
    IN: [{ id: 'MH', name: 'Maharashtra' }, { id: 'DL', name: 'Delhi' }],
    US: [{ id: 'CA', name: 'California' }, { id: 'NY', name: 'New York' }]
  },
  cities: {
    MH: ['Mumbai', 'Pune', 'Nagpur'],
    DL: ['New Delhi', 'Dwarka'],
    CA: ['Los Angeles', 'San Francisco'],
    NY: ['New York City', 'Buffalo']
  }
};

function setError(m) { document.getElementById('error').textContent = m; }

function loadStates() {
  var country = document.getElementById('country').value;
  var stateEl = document.getElementById('state');
  var cityEl = document.getElementById('city');
  setError('');
  stateEl.innerHTML = '<option>Loading...</option>';
  stateEl.disabled = true;
  cityEl.innerHTML = '<option>-- Select City --</option>';
  cityEl.disabled = true;

  if (!country) { stateEl.innerHTML = '<option>-- Select State --</option>'; return; }

  var xhr = new XMLHttpRequest();
  xhr.open('GET', 'https://jsonplaceholder.typicode.com/todos/1');
  xhr.onload = function () {
    var states = mockData.states[country];
    if (states) {
      stateEl.innerHTML = '<option value="">-- Select State --</option>' +
        states.map(function (s) {
```



```

        return '<option value="" + s.id + ">' + s.name + '</option>';
    }).join('');
    stateEl.disabled = false;
} else {
    setError('No states found.');
```

stateEl.innerHTML = '<option>-- Select State --</option>';

```

    }
};
xhr.onerror = function () { setError('Network error.');
```

xhr.send();

```

}

function loadCities() {
    var state = document.getElementById('state').value;
    var cityEl = document.getElementById('city');
```

setError('');

```

    cityEl.innerHTML = '<option>Loading...</option>';
    cityEl.disabled = true;

    if (!state) { cityEl.innerHTML = '<option>-- Select City --</option>'; return; }

    var xhr = new XMLHttpRequest();
    xhr.open('GET', 'https://jsonplaceholder.typicode.com/todos/1');
```

xhr.onload = function () {

```

    var cities = mockData.cities[state];
    if (cities) {
        cityEl.innerHTML = '<option value="">-- Select City --</option>' +
            cities.map(function (c) {
                return '<option value="" + c + ">' + c + '</option>';
            }).join('');
```

cityEl.disabled = false;

```

    } else {
        setError('No cities found.');
```

cityEl.innerHTML = '<option>-- Select City --</option>';

```

    }
};
xhr.onerror = function () { setError('Network error.');
```

xhr.send();

```

}
</script>
</body>
</html>
```

Assignment 24: Login Form with AJAX (Form Handling + Validation)

```
<!DOCTYPE html>
<html>
<head><title>Login</title>
<style>
  body      { font-family: Arial; max-width: 350px; margin: 80px auto;
              padding: 20px; border: 1px solid #ddd; border-radius: 8px; }
  input     { padding: 8px; width: 100%; margin: 6px 0; box-sizing: border-box; }
  button    { padding: 10px; width: 100%; background: #4a90e2;
              color: white; border: none; cursor: pointer; }
  button:disabled { background: #aaa; }
  #msg      { margin-top: 10px; padding: 8px; border-radius: 4px; display: none; }
  .success  { background: #d4edda; color: #155724; }
  .error-box { background: #f8d7da; color: #721c24; }
  #spinner  { display: none; text-align: center; color: gray; margin: 6px; }
</style>
</head>
<body>
<h2>Login</h2>
<input type="email" id="email" placeholder="Email">
<input type="password" id="password" placeholder="Password">
<div id="spinner">Logging in...</div>
<button id="loginBtn" onclick="login()">Login</button>
<div id="msg"></div>

<script>
function showMsg(text, cls) {
  var msg = document.getElementById('msg');
  msg.textContent = text;
  msg.className = cls;
  msg.style.display = 'block';
}

function login() {
  var email = document.getElementById('email').value.trim();
  var password = document.getElementById('password').value.trim();
  var btn = document.getElementById('loginBtn');
  var spinner = document.getElementById('spinner');

  document.getElementById('msg').style.display = 'none';

  // Client-side validation
  if (!email || !password) { showMsg('All fields are required.', 'error-box'); return; }
  if (!/^\S+@\S+\.\S+$/.test(email)) { showMsg('Invalid email format.', 'error-box'); return; }
  if (password.length < 6) { showMsg('Password min 6 chars.', 'error-box'); return; }

  btn.disabled = true;
  spinner.style.display = 'block';

  var xhr = new XMLHttpRequest();
  xhr.open('POST', 'https://jsonplaceholder.typicode.com/posts');
  xhr.setRequestHeader('Content-Type', 'application/json');

  xhr.onload = function () {
    spinner.style.display = 'none';
    btn.disabled = false;
    if (xhr.status === 201)
      showMsg('Login successful! Welcome back.', 'success');
    else
      showMsg('Login failed. Please try again.', 'error-box');
  };

  xhr.onerror = function () {
```

```
    spinner.style.display = 'none';
    btn.disabled = false;
    showMsg('Network error. Try again.', 'error-box');
};

    xhr.send(JSON.stringify({ email: email, password: password }));
}
</script>
</body>
</html>
```

Assignment 25: Comment System (AJAX + DOM Updates with Fade-In)

```
<!DOCTYPE html>
<html>
<head><title>Comment System</title>
<style>
  body      { font-family: Arial; max-width: 500px; margin: 50px auto; padding: 20px; }
  textarea  { width: 100%; padding: 8px; box-sizing: border-box; height: 80px; }
  button    { padding: 8px 16px; background: #4a90e2; color: white; border: none; cursor: pointer; }
  .comment  { padding: 10px; margin: 8px 0; background: #f5f5f5;
              border-left: 3px solid #4a90e2; opacity: 0; }

  #error     { color: red; }
  #spinner   { display: none; color: gray; }
</style>
</head>
<body>
<h2>Comments</h2>
<textarea id="commentInput" placeholder="Write a comment..."></textarea>
<button onclick="postComment()">Post Comment</button>
<div id="spinner">Loading...</div>
<div id="error"></div>
<div id="commentList"></div>

<script>
function showSpinner(s) { document.getElementById('spinner').style.display = s ? 'block' : 'none'; }
function setError(m)    { document.getElementById('error').textContent = m; }

// CSS-free fade-in using JS opacity steps
function fadeIn(el) {
  el.style.opacity = 0;
  el.style.display = 'block';
  var op = 0;
  var t = setInterval(function () {
    if (op >= 1) { clearInterval(t); return; }
    op += 0.1;
    el.style.opacity = op;
  }, 30);
}

function addComment(name, body) {
  var div = document.createElement('div');
  div.className = 'comment';
  div.innerHTML = '<strong>' + name + '</strong><p>' + body + '</p>';
  document.getElementById('commentList').prepend(div);
  fadeIn(div);
}

function fetchComments() {
  showSpinner(true);
  var xhr = new XMLHttpRequest();
  xhr.open('GET', 'https://jsonplaceholder.typicode.com/comments?_limit=5');
  xhr.onload = function () {
    showSpinner(false);
    if (xhr.status === 200)
      JSON.parse(xhr.responseText).forEach(function (c) { addComment(c.name, c.body); });
    else
      setError('Failed to load comments.');
```

```
  };
  xhr.onerror = function () { showSpinner(false); setError('Network error.');
```

```
  };
  xhr.send();
}

function postComment() {
```

```
var body = document.getElementById('commentInput').value.trim();
setError('');
if (!body) { setError('Comment cannot be empty.');
```

```
return; }

showSpinner(true);
var xhr = new XMLHttpRequest();
xhr.open('POST', 'https://jsonplaceholder.typicode.com/comments');
xhr.setRequestHeader('Content-Type', 'application/json');
xhr.onload = function () {
    showSpinner(false);
    if (xhr.status === 201) {
        addComment('You', body);
        document.getElementById('commentInput').value = '';
    } else {
        setError('Failed to post comment.');
```

```
    }
};
xhr.onerror = function () { showSpinner(false); setError('Network error.');
```

```
};
xhr.send(JSON.stringify({ name: 'You', body: body, email: 'u@x.com', postId: 1 }));
}

fetchComments();
</script>
</body>
</html>
```

Assignment 26: Animated News Feed (jQuery + AJAX)

jQuery CDN used. Uses JSONPlaceholder /posts as mock news data.

```
<!DOCTYPE html>
<html>
<head><title>News Feed</title>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<style>
  body      { font-family: Arial; max-width: 500px; margin: 50px auto; padding: 20px; }
  .news-item { padding: 12px; margin: 6px 0; background: #f0f4ff;
               border-left: 4px solid #4a90e2; display: none; }
  button     { padding: 8px 16px; background: #4a90e2; color: white;
               border: none; cursor: pointer; margin-bottom: 10px; }
  #spinner   { display: none; color: gray; }
  #error     { color: red; }
</style>
</head>
<body>
<h2>News Feed</h2>
<button id="refreshBtn" onclick="loadNews()">Refresh News</button>
<div id="spinner">Loading...</div>
<div id="error"></div>
<div id="newsFeed"></div>

<script>
function loadNews() {
  var feed = $('#newsFeed');
  feed.empty();
  $('#error').text('');
  $('#refreshBtn').prop('disabled', true);
  $('#spinner').show();

  $.ajax({
    url: 'https://jsonplaceholder.typicode.com/posts?_limit=8',
    method: 'GET',
    success: function (data) {
      $('#spinner').hide();
      $('#refreshBtn').prop('disabled', false);

      $.each(data, function (i, item) {
        var div = $('<div class="news-item"></div>').html(
          '<strong>' + item.title + '</strong>' +
          '<p>' + item.body.substring(0, 80) + '...</p>'
        );
        feed.append(div);
        // staggered slideDown animation
        setTimeout(function () { div.slideDown(400); }, i * 150);
      });
    },
    error: function () {
      $('#spinner').hide();
      $('#refreshBtn').prop('disabled', false);
      $('#error').text('Failed to fetch news. Please try again.');
```

Assignment 27: Shopping Cart (AJAX + jQuery Animations)

```
<!DOCTYPE html>
<html>
<head><title>Shopping Cart</title>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<style>
  body      { font-family: Arial; max-width: 600px; margin: 30px auto; padding: 20px; }
  .product  { padding: 10px; margin: 5px 0; background: #f9f9f9;
              display: flex; justify-content: space-between; align-items: center; }
  .cart-item { padding: 8px; margin: 4px 0; background: #e8f4fd;
              display: flex; justify-content: space-between; align-items: center; }
  .add-btn   { background: #28a745; color: white; border: none; padding: 6px 12px; cursor: pointer; }
  .rem-btn   { background: #dc3545; color: white; border: none; padding: 4px 10px; cursor: pointer; }
  #total     { font-weight: bold; margin-top: 10px; }
  #spinner   { display: none; color: gray; }
</style>
</head>
<body>
<h2>Products</h2>
<div id="spinner">Loading products...</div>
<div id="products"></div>
<hr>
<h3>Cart</h3>
<div id="cartItems"></div>
<div id="total">Total: Rs. 0</div>

<script>
var cart = {};

function loadProducts() {
  $('#spinner').show();
  $.ajax({
    url: 'https://jsonplaceholder.typicode.com/users?_limit=5',
    success: function (data) {
      $('#spinner').hide();
      $.each(data, function (i, item) {
        var price = (i + 1) * 99;          // deterministic mock price
        item.price = price;
        $('#products').append(
          '<div class="product">' +
            '<span>' + item.name + ' - Rs. ' + price + '</span>' +
            '<button class="add-btn" onclick="addToCart(' + item.id + ', "' +
              item.name + '", ' + price + ')">Add to Cart</button>' +
          '</div>'
        );
      });
    },
    error: function () { $('#spinner').hide(); alert('Failed to load products.')}
  });
}

function addToCart(id, name, price) {
  $.ajax({
    url: 'https://jsonplaceholder.typicode.com/posts',
    method: 'POST',
    contentType: 'application/json',
    data: JSON.stringify({ id: id, name: name, price: price }),
    success: function () {
      if (cart[id]) {
        cart[id].qty++;
        $('#cart-' + id + ' .qty').text('x' + cart[id].qty);
      } else {

```

```

    cart[id] = { name: name, price: price, qty: 1 };
    var row = $(
        '<div class="cart-item" id="cart-' + id + '">' +
        '<span>' + name + ' <span class="qty">x1</span></span>' +
        '<span>Rs. ' + price +
        ' <button class="rem-btn" onclick="removeFromCart(' + id + ')">Remove</button>' +
        '</span>' +
        '</div>'
    );
    $('#cartItems').append(row);
}
updateTotal();
},
error: function () { alert('Failed to add item.'); }
});
}

function removeFromCart(id) {
    $('#cart-' + id).fadeOut(400, function () { $(this).remove(); });
    delete cart[id];
    updateTotal();
}

function updateTotal() {
    var total = 0;
    $.each(cart, function (k, v) { total += v.price * v.qty; });
    $('#total').text('Total: Rs. ' + total);
}

loadProducts();
</script>
</body>
</html>

```


Assignment 28: Infinite Scroll (jQuery + AJAX)

```
<!DOCTYPE html>
<html>
<head><title>Infinite Scroll</title>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<style>
  body    { font-family: Arial; max-width: 500px; margin: 30px auto; padding: 20px; }
  .post   { padding: 12px; margin: 8px 0; background: #f9f9f9;
            border-left: 4px solid #4a90e2; display: none; }
  #loading{ text-align: center; padding: 10px; display: none; color: gray; }
  #end     { text-align: center; color: #999; display: none; }
</style>
</head>
<body>
<h2>Posts (Scroll to Load More)</h2>
<div id="posts"></div>
<div id="loading">Loading more posts...</div>
<div id="end">No more posts</div>

<script>
var page    = 1;
var limit   = 5;
var loading = false;
var done    = false;

function loadPosts() {
  if (loading || done) return;
  loading = true;
  $('#loading').show();

  $.ajax({
    url: 'https://jsonplaceholder.typicode.com/posts?_page=' + page + '&_limit=' + limit,
    success: function (data) {
      $('#loading').hide();
      loading = false;

      if (data.length === 0) { done = true; $('#end').show(); return; }

      $.each(data, function (i, post) {
        var div = $('<div class="post"></div>').html(
          '<strong>' + post.title + '</strong><p>' + post.body + '</p>'
        );
        $('#posts').append(div);
        div.fadeIn(400);
      });
      page++;
    },
    error: function () {
      $('#loading').hide();
      loading = false;
      alert('Failed to load posts.');
```

```
</script>  
</body>  
</html>
```

Assignment 29: Interactive Poll System (AJAX + Real-Time Updates)

Prevents multiple votes using localStorage. Uses slideUp/slideDown for result animation.

```
<!DOCTYPE html>
<html>
<head><title>Poll System</title>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<style>
  body { font-family: Arial; max-width: 450px; margin: 50px auto; padding: 20px; }
  .option { padding: 10px; margin: 6px 0; background: #f0f4ff;
    cursor: pointer; display: flex; justify-content: space-between; }
  .option:hover { background: #dce9ff; }
  .option.voted { background: #d4edda; cursor: default; }
  .bar { height: 8px; background: #4a90e2; margin-bottom: 6px; transition: width 0.5s; }
  #result { margin-top: 10px; color: green; display: none; }
  #error { color: red; }
  #spinner{ display: none; color: gray; }
</style>
</head>
<body>
<h2>Poll: Best Programming Language?</h2>
<div id="spinner">Loading...</div>
<div id="error"></div>
<div id="pollOptions"></div>
<div id="result"></div>

<script>
// Mock poll data
var poll = {
  options: [
    { id: 1, label: 'JavaScript', votes: 34 },
    { id: 2, label: 'Python',      votes: 52 },
    { id: 3, label: 'Java',        votes: 18 },
    { id: 4, label: 'C++',         votes: 22 }
  ]
};

var voted = localStorage.getItem('poll_voted'); // prevent double vote

function renderPoll() {
  var total = 0;
  $.each(poll.options, function (i, o) { total += o.votes; });

  var html = '';
  $.each(poll.options, function (i, opt) {
    var pct = total ? Math.round(opt.votes / total * 100) : 0;
    var isVoted = (voted == opt.id);
    var clickFn = isVoted ? '' : 'vote(' + opt.id + ')';
    html +=
      '<div class="option" + (isVoted ? ' voted' : '') + '"' +
        (clickFn ? ' onclick="' + clickFn + '"' : '') + '>' +
        '<span>' + opt.label + (isVoted ? ' (Your vote)' : '') + '</span>' +
        '<span>' + opt.votes + ' votes (' + pct + '%)</span>' +
      '</div>' +
      '<div class="bar" style="width:' + pct + '%"></div>';
  });

  $('#pollOptions').html(html);
  if (voted) $('#result').text('You have already voted!').show();
}

function vote(optId) {
  if (voted) { $('#error').text('You have already voted!'); return; }
```

```

$('#spinner').show();
$('#error').text('');

$.ajax({
  url: 'https://jsonplaceholder.typicode.com/posts',
  method: 'POST',
  contentType: 'application/json',
  data: JSON.stringify({ optionId: optId }),
  success: function () {
    $('#spinner').hide();

    // Animate: slideUp → update data → slideDown
    $('#pollOptions').slideUp(300, function () {
      var opt = null;
      $.each(poll.options, function (i, o) { if (o.id === optId) opt = o; });
      opt.votes++;
      voted = optId;
      localStorage.setItem('poll_voted', optId);
      renderPoll();
      $('#pollOptions').slideDown(300);
      $('#result').text('Vote cast for ' + opt.label + '!').show();
    });
  },
  error: function () {
    $('#spinner').hide();
    $('#error').text('Failed to submit vote. Try again.');
```

```

  }
});
}

```

```

// Simulate initial load
$('#spinner').show();
setTimeout(function () { $('#spinner').hide(); renderPoll(); }, 400);
</script>
</body>
</html>

```